

Lab 8B – Practical REST API Integration Turning API Data into Useful Apps

1. Lab Title & Overview

Overview:

In this lab, students will build a small but practical Flutter app that consumes a real-world public REST API and uses the data for a meaningful purpose, not just as a simple list.

Students can choose **one** of the following scenarios (or more, for extra credit):

1. **Weather Companion App** – Use a weather API to help users decide what to do or what to wear.
2. **Movie & TV Explorer** – Use a movie API to help users find something interesting to watch.
3. *(Optional)* **Currency Rate Helper** – Use an exchange rate API to support simple travel or shopping decisions.

The lab reinforces everything from Module 8:

- HTTP + REST
- GET requests
- JSON parsing → model classes
- FutureBuilder + ListView / UI
- Loading & error states
- Service Layer pattern

2. Learning Objectives

After completing this lab, students will be able to:

- Integrate a **real public REST API** into a Flutter app.
- Convert JSON responses into **Dart models** that are easy to use.
- Design a **purpose-driven UI** where API data helps users make decisions.
- Implement loading, empty, and error states with a **good UX**.
- Organize networking code with a **service layer** for better structure.

3. Scenarios (Choose at least 1)

3.1 Scenario A – Weather Companion App

Goal of the app (real-world use):

Help the user answer questions like:

- “Do I need an umbrella today?”
- “Is it a good day for outdoor activities?”
- “How hot/cold will it be this afternoon?”

Suggested Features:

- Let the user:
 - Search or choose a city/region
 - View **current temperature, weather description**, and maybe an **icon**
 - See short info like “Feels like...”, “Wind speed...”, or “Humidity...”
- Use weather information to show:
 - A simple recommendation:
 - “Take an umbrella” / “No umbrella needed”
 - “Too hot for outdoor sports” / “Nice weather for a walk”

API Examples (for documentation purposes):

- Open weather APIs (e.g., Open-Meteo, OpenWeatherMap, etc.)
(Teacher can suggest 1–2 concrete endpoints in class.)
-

3.2 Scenario B – Movie & TV Explorer App

Goal of the app (real-world use):

Help the user quickly discover something to watch tonight.

Suggested Features:

- Fetch a list of:
 - Trending movies

- Popular TV shows
- Or top-rated content
- Let the user:
 - Scroll through a list of titles + poster + rating
 - Tap on an item to see more details (overview, year, rating)
- Optional:
 - Add a simple “Watch later” / “Favorites” marker
 - Filter by rating or genre (client-side filter)

API Examples:

- Movie DB-style APIs (e.g., TMDb)
(Teacher may provide a shared API key or use a demo endpoint in class)
-

3.3 (Optional) Scenario C – Currency Rate Helper

Goal of the app (real-world use):

Support basic decisions like:

- “If I have X VND, how much is that in USD?”
- “Is this price high compared to another currency?”

Suggested Features:

- Fetch latest exchange rates for a base currency (e.g., USD or EUR)
- Let the user:
 - Input an amount
 - Choose a target currency
 - See converted value
- Optional:
 - Show a short hint:
 - “Good time to buy” / “Rate is higher than average” (simple heuristic)

API Examples:

- Free exchange rate APIs (e.g., `exchangerate.host`, `Frankfurter`, etc.)
-

4. Functional Requirements (for all scenarios)

Regardless of the scenario chosen, the app must:

1. Call a real REST API

- Use `http` package
- Handle network errors gracefully

2. Parse JSON into model classes

- Use at least one Dart model class with fields + `fromJson`
- Avoid using raw dynamic maps all over the UI

3. Display data with a meaningful UI

- Use cards, tiles, or sections that clearly show important information
- Not just a debug `Text(response.body)`

4. Handle loading & error states

- Show a loading indicator while waiting for API
- Show an error message and a **Retry** action if the call fails
- Handle empty or unexpected responses

5. Use a Service Layer

- Separate API calls into a dedicated service class
 - Screens/widgets should call service methods, not `http` directly
-

5. Technical Requirements

- Flutter project (can be reused from Lab 8 or created new)
- Use `http` package for all network calls

- Use Future / async/await for asynchronous operations
- Use FutureBuilder or equivalent pattern for binding API data to UI
- Use at least:
 - 1 model class
 - 1 service class
 - 1 main screen

(Optional but recommended):

- Detail screen
- Basic form (for search, filters, or user input)

6. Suggested Lab Steps (For Students)

Step 1 – Choose a Scenario

Pick **Weather**, **Movie**, or **Currency** (or another approved by the instructor).

Step 2 – Explore the API

- Read the API docs
- Identify:
 - Base URL
 - Endpoint
 - Required parameters
 - JSON response structure

Step 3 – Design the UI (Paper or Figma-level, simple)

- Decide:
 - What information you want to show
 - How it helps the user make a decision
 - Where loading / errors will appear

Step 4 – Define Model Classes

- Identify key fields from JSON (id, title, temp, rate, etc.)
- Create model structure on paper or in code (but lab document không cần code).

Step 5 – Plan the Service Layer

- Decide function names like:
 - fetchWeatherForCity
 - fetchTrendingMovies
 - fetchExchangeRates
- Clarify input → output:
 - Input: location / query / base currency
 - Output: List<Model> or a single Model

Step 6 – Integrate Into UI

- Use FutureBuilder to:
 - call the service
 - show loading/error/data
- Map model data → UI widgets (cards, list tiles, text, icons).

Step 7 – Add a “Purpose-driven element”

- For Weather: “recommendation text” (bring umbrella?, good for running?)
- For Movies: highlight top N choices, or mark favorites
- For Currency: show converted value + simple hint

7. Deliverables & Submission

Students must submit:

1. **App description (short):**
 - Which scenario they chose

- What problem their app helps the user solve

2. Screenshots:

- Loading state
- Error state (e.g., when network off or invalid URL)
- Successful data display screen
- Any optional features (favorites, form, recommendation text, etc.)

3. Source code (project or repository link)

- main.dart
- model file(s)
- service file(s)
- main screen(s)

(Trong file lab, bạn chỉ mô tả yêu cầu; code sinh viên tự làm như các lab trước.)

8. Evaluation Criteria (for teacher)

You can reuse khung chấm của Lab 8, tinh chỉnh nhẹ:

Category	Weight	Description
API Integration	25%	Correct use of HTTP + endpoint, API actually called
JSON & Model Design	20%	Good mapping JSON → Dart models
UI & UX	20%	Data clearly presented, purpose-driven, not just raw text
Async Handling	20%	Loading, error, retry states, no crashes
Architecture	10%	Service layer, separation of concerns
Creativity	5%	How meaningful/interesting the scenario & decisions are